Problem Statement

Date	2-October- 2026
TeamID	6ab46978b5170a9b58b3f461
ProjectName	**Implement Client Script & UI Policy (Incident)**
Maximum Marks	3 Marks

Implement Client Script & UI Policy (Incident)

Implement Client Script & UI Policy (Incident)

ServiceNow Incident Management is used to record, manage, assign, update, and track incidents throughout their lifecycle. For an Incident record to be processed correctly, important information must be entered consistently and according to the conditions associated with the incident. When field behavior depends only on manual user input, there can be a risk of incomplete or inconsistent information. The project **"Implement Client Script & UI Policy (Incident)"** addresses this requirement by introducing conditional controls and client-side validation on the ServiceNow Incident table.

The main problem identified for the project is the need to control Incident fields dynamically according to the **Impact** value. In particular, when an Incident has **Impact = 1 – High**, certain fields require special handling. The project therefore defines a **High Impact Control UI Policy** for the Incident table. The UI Policy is activated for the High Impact condition and is designed to control the behavior of fields on the Incident form. The configuration also uses the **Reverse if false** option so that the policy-controlled behavior can be reversed when the specified condition is no longer satisfied. Implement Client Script & UI Po…

One of the important problems addressed is the requirement for an appropriate **Assignment Group** for High Impact incidents. Assignment Group is important for directing an Incident to the appropriate support or responsible group. To address this requirement, the UI Policy Action is configured so that the **Assignment Group field becomes mandatory** when the High Impact condition is satisfied. This helps ensure that the required assignment information is available before the Incident is processed. Implement Client Script & UI Po…

Another problem concerns the control of the **Urgency** field. For a High Impact Incident, the project requires Urgency to be controlled rather than freely modified by the user. The UI Policy Action therefore configures the **Urgency field as read-only** under the High Impact condition. This prevents inappropriate manual modification while the policy is active. When the condition becomes false, the configured reverse behavior allows the normal field behavior to be restored. Implement Client Script & UI Po…

The project also recognizes that UI Policies alone do not address every required behavior. Some requirements involve automatic value changes and validation during user interaction. Therefore,

Problem Statement

Client Scripts are introduced as a complementary mechanism. The combination of UI Policies and Client Scripts provides a structured approach for controlling Incident form behavior and improving the consistency of information entered into the Incident record.

A further problem is the need to automatically handle the **Urgency** value when the Impact field changes. The project implements an **onChange Client Script** that responds to changes in the relevant field. When the new Impact value is High, the script sets Urgency to High and displays an informational message to the user. This reduces the need for the user to manually enter the corresponding value and helps maintain consistency between the Impact and Urgency fields. Implement Client Script & UI Po…

The project also addresses the problem of incomplete information at the time an Incident is submitted. In particular, a High Impact Incident should not be submitted without an **Assigned To** value. An **onSubmit Client Script** is therefore used to check the Incident before submission. If Impact is High and Assigned To is empty, the script displays an error message and prevents the record from being submitted. If the required information is present, the submission is allowed to continue. Implement Client Script & UI Po…

Another identified requirement is the control of direct editing through the Incident list view. Users may otherwise attempt to change the **State** field directly using list editing. The project therefore includes an **onCellEdit Client Script** to control this behavior. When a user attempts to modify State through list editing, the configured script blocks the update and instructs the user to open the Incident form. This encourages State changes to be performed through the controlled form interface rather than through direct list editing. Implement Client Script & UI Po…

The problem statement therefore covers several related requirements: conditional field control, mandatory information, automatic field updates, submission validation, and controlled list editing. These requirements are addressed through the combined implementation of the **High Impact Control UI Policy**, UI Policy Actions, and three Client Script types—**onChange, onSubmit, and onCellEdit**.

The project also requires systematic testing to confirm that the configured controls behave as intended. The testing process includes checking High Impact behavior, validating the Assigned To requirement, confirming successful submission after the required information is entered, checking the reverse condition, and verifying that direct State editing from the list is controlled. The source project specifically describes testing the mandatory validation and successful submission behavior for High Impact incidents. Implement Client Script & UI Po…

Problem Statement Summary

In summary, the problem addressed by this project is the need for **consistent, controlled, and validated Incident data entry in ServiceNow**. The project solves this by using UI Policies to dynamically control fields and Client Scripts to provide automatic updates and validation. The resulting configuration is intended to reduce incomplete Incident information, control field behavior according to Impact, validate required information before submission, and restrict inappropriate list-based editing. Thus, the project provides a practical ServiceNow configuration approach for maintaining reliable and consistent Incident records.